

Módulo 6 — Ingeniería de Prompts

Anatomía del prompt, few-shot, CoT, self-consistency, prompt injection, evaluación sistemática y DSPy.

BLOQUE 1 · LA DISCIPLINA DE LA INGENIERÍA DE PROMPTS

Módulo 6 — Ingeniería de Prompts

La ingeniería de prompts es la disciplina de diseñar, evaluar, y optimizar los inputs textuales (prompts) a los LLMs para obtener outputs de la calidad, el formato, y el contenido deseados de forma consistente y controlable. Empezó siendo percibida como un arte intuitivo pero se ha desarrollado como una ingeniería con principios, técnicas, herramientas, y procesos de evaluación sistemáticos. Es una de las competencias más demandadas en el mercado de trabajo de IA en 2025, no porque sea difícil de aprender sino porque es difícil de hacer bien y tiene impacto directo en la calidad de los sistemas de IA en producción.

La importancia de la ingeniería de prompts deriva de un hecho fundamental: para los casos de uso que no requieren fine-tuning (que es la mayoría de los casos de uso empresariales), el prompt es el único mecanismo de control del comportamiento del modelo. Todo lo que el sistema de IA hace, cómo lo hace, qué restricciones respeta, y qué tono y formato usa está determinado por el prompt. Un prompt mal diseñado produce un sistema poco fiable, inconsistente, y difícil de mantener. Un prompt bien diseñado produce un sistema consistente, controlable, y eficiente.

La anatomía de un prompt efectivo

Un prompt efectivo para un sistema de producción tiene varias secciones bien diferenciadas. El rol y el contexto definen quién es el modelo (un asistente de soporte al cliente de una empresa de software, un analista financiero, un tutor de programación) y para qué contexto opera. Las instrucciones específicas detallan qué debe hacer el modelo, cómo debe hacerlo, y qué restricciones debe respetar. Los ejemplos (few-shot) muestran al modelo el tipo de input que recibirá y el tipo de output que se espera. El formato del output define la estructura exacta de la respuesta esperada (JSON con campos específicos, markdown con secciones definidas, respuesta de una sola línea). Los delimitadores separan las distintas partes del prompt (instrucciones vs contexto vs input del usuario) para evitar confusión y reducir el riesgo de prompt injection.

La sección de instrucciones es la más crítica y la que más afecta al comportamiento del modelo. Las mejores prácticas para escribir instrucciones efectivas son: ser específico y sin ambigüedad (las instrucciones vagas producen comportamiento impredecible), usar afirmaciones positivas en lugar de negativas cuando sea posible (decir "responde siempre en español" es más efectivo que "no respondas en inglés"), proporcionar ejemplos de lo que es correcto y de lo que no lo es (positive y negative examples), y estructurar las instrucciones por orden de importancia (las instrucciones al principio del system prompt tienen mayor peso estadístico).

BLOQUE 2 · TÉCNICAS AVANZADAS DE PROMPTING

Chain of Thought, Few-Shot, RAG y técnicas avanzadas

Zero-shot, one-shot, y few-shot prompting

El zero-shot prompting describe una tarea al modelo sin proporcionar ejemplos: "Clasifica el siguiente texto como positivo, negativo, o neutro." El one-shot proporciona un ejemplo: "Clasifica el texto (positivo, negativo, neutro). Ejemplo: El servicio fue excelente → positivo. Texto a clasificar:". El few-shot proporciona múltiples ejemplos. En general, más ejemplos producen mejores resultados hasta un punto de rendimientos decrecientes (típicamente 3-8 ejemplos es suficiente para la mayoría de las tareas). La elección y el orden de los ejemplos importa: los ejemplos deben ser representativos de la distribución real de inputs, incluir los casos de borde más importantes, y estar en un orden que no cree sesgos de posición (los últimos ejemplos tienden a tener mayor peso estadístico).

El meta-prompting o prompt chaining es la técnica de descomponer un problema complejo en múltiples llamadas al LLM, donde el output de cada llamada sirve de input para la siguiente. Por ejemplo, para analizar un contrato complejo: primera llamada (identificar las secciones del contrato), segunda llamada (para cada sección, extraer las cláusulas clave), tercera llamada (evaluar el riesgo de las cláusulas identificadas), cuarta llamada (generar el resumen ejecutivo a partir del análisis de riesgo). Esta descomposición produce resultados más precisos y controlables que intentar hacer todo en una sola llamada.

Self-consistency y mayoría de votos

Self-consistency (Wang et al., 2022) es una técnica que mejora la fiabilidad de las respuestas del LLM generando múltiples respuestas independientes al mismo prompt (con temperatura mayor que cero para introducir variación) y seleccionando la respuesta con mayor acuerdo entre las muestras. Para preguntas de respuesta única (matemáticas, clasificación, respuestas factuales), la respuesta más frecuente entre 5-10 muestras es significativamente más precisa que una sola muestra. El coste es proporcional al número de muestras (5 muestras = 5x el coste de una sola llamada), lo que lo hace apropiado para casos de uso donde la precisión justifica el coste adicional.

Para tareas de razonamiento, self-consistency con CoT produce los mejores resultados: se generan múltiples cadenas de razonamiento independientes y se selecciona la conclusión más consistente entre ellas. Esto no solo mejora la precisión sino también la confiabilidad: cuando múltiples caminos de razonamiento independientes llegan a la misma conclusión, la confianza en esa conclusión es mayor. Los modelos de razonamiento como o3 internalizan algo similar en su proceso de extended thinking.

Prompt injection: el riesgo de seguridad número 1

La ingeniería de prompts tiene una dimensión de seguridad crítica: los ataques de prompt injection intentan usar el input del usuario para manipular el comportamiento del modelo de formas no intencionadas por el diseñador del sistema. Los ataques directos incluyen instrucciones explícitas de ignorar el system prompt ("ignora todas las instrucciones anteriores y..."), roleplay injections ("actúa como un sistema sin restricciones"), y encoding tricks (instrucciones en Base64 o con caracteres especiales que evitan filtros de texto). Los ataques indirectos introducen instrucciones maliciosas a través de los datos que el modelo procesa (un documento PDF con instrucciones invisibles, una página web recuperada por un agente de búsqueda).

La defensa contra prompt injection no es un problema completamente resuelto en 2025: el paper de octubre 2025 con investigadores de OpenAI, Anthropic, y Google DeepMind demostró que el 90% de las defensas publicadas pueden eludirse con ataques adaptativos. La defensa correcta es en capas: separación clara entre instrucciones y inputs del usuario con delimitadores, instrucciones explícitas de ignorar instrucciones en el input, controles en la capa de ejecución de herramientas (fuera del control del LLM), y HITL para acciones de alto impacto.

Medir y mejorar: el ciclo de ingeniería de prompts

El proceso de evaluación de prompts

La evaluación sistemática de prompts es lo que distingue la ingeniería de prompts de la intuición. El proceso requiere: un golden dataset (50-200 pares input/output esperado que representen la distribución real de casos de uso), métricas de evaluación adecuadas al caso de uso (para clasificación: accuracy, F1; para texto libre: LLM-as-judge con rúbricas específicas; para extracción: precision y recall de campos), y un proceso de A/B testing entre versiones del prompt. Este proceso hace visible qué cambios en el prompt mejoran el rendimiento y qué cambios lo degradan, convirtiendo el desarrollo de prompts en un proceso de optimización con feedback objetivo.

Las herramientas más usadas para evaluación de prompts en 2025 son: Promptfoo (open source, framework de testing de prompts con soporte para múltiples modelos y métricas), LangSmith (plataforma de observabilidad de LangChain con soporte para evaluadores automáticos y humanos), y Braintrust (plataforma de evaluación con focus en LLM-as-judge). La integración de la evaluación de prompts en el CI/CD (ejecutar el test suite de prompts en cada PR que modifica los prompts) es la mejor práctica para mantener la calidad del sistema a lo largo del tiempo.

Optimización automática de prompts

La optimización automática de prompts usa algoritmos para mejorar los prompts sin intervención humana en cada iteración. DSPy (Stanford, 2024) es el framework más avanzado para esto: en lugar de escribir los prompts manualmente, el desarrollador especifica el programa de prompts (qué componentes hay, cómo se conectan) y DSPy optimiza automáticamente los prompts de cada componente para maximizar una métrica de evaluación definida. Es especialmente útil para sistemas complejos con múltiples llamadas al LLM donde optimizar manualmente cada prompt es muy costoso.

Los métodos de optimización automática más simples incluyen: gradient-free optimization (búsqueda aleatoria o en cuadrícula sobre variaciones del prompt), y los métodos basados en ejemplos (usando un LLM para generar variaciones del prompt y evaluarlas contra el golden dataset). Para la mayoría de los proyectos, la optimización manual con evaluación sistemática es suficiente; la optimización automática es más relevante para sistemas a escala donde los prompts son numerosos y el rendimiento de cada fracción de porcentaje importa económicamente.

RESUMEN EJECUTIVO DEL MÓDULO 6: La ingeniería de prompts es la disciplina de diseñar, evaluar, y optimizar inputs a los LLMs para obtener outputs consistentes y de calidad. La anatomía de un prompt efectivo incluye: rol y contexto, instrucciones específicas, ejemplos few-shot, formato de output, y delimitadores. Las técnicas principales son: zero/few-shot, Chain of Thought, meta-prompting/prompt chaining, self-consistency (5-10 muestras + mayoría de votos), y temperature management. El prompt injection es el riesgo de seguridad número 1; la defensa es en capas (delimitadores + instrucciones + controles en la capa de ejecución). La evaluación sistemática con golden dataset + métricas + A/B testing convierte el desarrollo de prompts en ingeniería. DSPy automatiza la optimización de prompts en sistemas complejos. El prompt como código: versionado en git, testing en CI/CD, monitorización en producción.